

VidSnap — Technical Architecture & Maintenance Report

Version: 2.0 | Core Stack: FastAPI, Python 3.11, yt-dlp, FFmpeg | Date: August 2026

1. Executive Summary & Core Requirements

VidSnap is a high-performance, multi-platform media downloader designed to fetch and stream video and audio content from YouTube, Facebook, and Instagram. The platform emphasizes instant browser-native downloads, full resolution support (360p up to 4K), accurate file size estimation, and robust server stability under high traffic.

Key Application Capabilities:

- **Multi-Platform Extraction:** Seamless support for YouTube, Facebook (Posts, Reels, Watch), and Instagram.
- **Adaptive Stream Merging:** Merges separate high-res video and audio tracks (1080p, 4K) into single playable MP4 files.
- **Chrome Download Manager Integration:** Direct CDN redirecting and streaming with live MB progress display.
- **Zero Storage Overhead:** Real-time FFmpeg pipe streaming prevents server disk bloat.

2. Core Technical Architecture & Fixes Implemented

During development, several critical challenges regarding adaptive media streaming, Windows process execution, and HTTP header limits were identified and permanently resolved. Below is the technical breakdown:

A. Adaptive Stream Merging (Audio + Video Fix)

Problem: High-resolution videos (1080p, 4K) on YouTube and Facebook use DASH adaptive streaming where video and audio are hosted as separate files. Downloading directly often resulted in audio-only output.

Architectural Solution: Implemented a dynamic format selector (`_build_format_selector`) that pairs requested video quality with the best available audio track using: `bestvideo[height<=1080]+bestaudio/b/best`. Additionally, explicit FFmpeg stream mapping `-map 0:v:0 -map 1:a:0` forces input 0 as video and input 1 as audio.

B. Chrome Native Download Manager Integration

Problem: Downloads previously completed internally on the server before transferring to Chrome, leaving the user with no native download progress tracker.

Solution: The endpoint `/api/stream` issues a `RedirectResponse` for single/pre-muxed streams (Facebook, Instagram, YouTube 360p/720p) direct to CDN, allowing Chrome's native Download Manager to immediately open and display live progress (e.g. `0.9 / 834 MB • 1 hour left`).

C. Windows Async Subprocess & Event Loop Handling

Problem: Running `asyncio.create_subprocess_exec` inside Starlette `StreamingResponse` iterators on Windows threw `NotImplementedError`.

Solution: Replaced async subprocess creation with standard `subprocess.Popen` and wrapped output chunk reads inside `asyncio.to_thread(process.stdout.read, 65536)`. This completely eliminates Windows event loop crashes.

D. Unicode & Emoji Filename Header Safety

Problem: Video titles containing emojis (👤, 📺) caused Starlette HTTP header encoding crashes (`UnicodeEncodeError: 'latin-1'`).

Solution: Applied RFC 5987 UTF-8 header encoding: filename*=UTF-8''{quoted_title} with an ASCII fallback parameter, ensuring 100% safety for all international characters.

E. Accurate File Size Calculation

Problem: UI table displayed video-only stream sizes (e.g. 13.3 MB for 1080p) while actual download was 33 MB.

Solution: Updated *fetch_info_sync* to add video_filesize + best_audio_filesize for DASH formats so the quality table renders exact total file sizes.

F. Facebook Share Link Mobile Fallback

Problem: Desktop Facebook share links (facebook.com/share/v/...) returned *Cannot parse data* due to Facebook login walls.

Solution: Added *normalize_url* helper and fallback handling to convert share links to m.facebook.com, bypassing login restrictions.

3. Essential Code Architecture Reference

yt-dlp Options Builder (downloader.py):

```
opts = { "format": format_selector, # bestvideo+bestaudio/b/best "outtmpl": output_template, "quiet":
True, "ffmpeg_location": os.path.dirname(find_ffmpeg()), "merge_output_format": "mp4", # Automatic
FFmpeg merging "postprocessors": [{"key": "FFmpegVideoConvertor", "preferredformat": "mp4"}] }
```

FFmpeg Live Streaming Command (downloader.py):

```
cmd = [ ffmpeg_bin, "-loglevel", "error", "-reconnect", "1", "-reconnect_at_eof", "1",
"-reconnect_streamed", "1", "-headers", v_headers, "-i", video_url, "-headers", a_headers, "-i",
audio_url, "-map", "0:v:0", "-map", "1:a:0", "-c:v", "copy" if "avc" in vcodec else "libx264", "-c:a",
"aac", "-b:a", "192k", "-movflags", "frag_keyframe+empty_moov+default_base_moof", "-f", "mp4", "pipe:1"
]
```

4. Backend API Routes Specification

Endpoint	Method	Description
/api/analyze	POST	Extracts metadata (title, thumb, duration, quality formats).
/api/stream	GET	Triggers Chrome Download Manager (redirects or FFmpeg stream).
/api/start-download	POST	Initiates background server download job.
/api/progress/{id}	GET (SSE)	Streams live SSE progress (% percentage, speed, ETA).
/api/fetch/{id}	GET	Retrieves completed job download file and cleans up temp.

5. Future Maintenance & Troubleshooting Guide

Issue Scenario	Root Cause	Action / Solution
Facebook links show <i>Cannot parse data</i>	Facebook updated page HTML / login wall active	Run pip install -U yt-dlp to get latest extractor fixes.
Private Video error	Video requires login or cookies	Provide cookies.txt in request parameters.
Chrome download stops mid-video	CDN socket timeout	Verify FFmpeg -reconnect 1 flags are present in <i>downloader.py</i> .
Server disk filling up	Temp files accumulated	Ensure <i>cleanup_job</i> is invoked after file fetch.

6. Production Deployment Instructions

1. Install Dependencies:

```
pip install fastapi uvicorn gunicorn yt-dlp pydantic reportlab
```

2. Run Gunicorn with 4 CPU Workers:

```
gunicorn -w 4 -k uvicorn.workers.UvicornWorker backend.main:app --bind 0.0.0.0:8000
```

3. Nginx Reverse Proxy Header Setup:

Ensure proxy_buffering off; and proxy_set_header X-Real-IP \$remote_addr; are set in Nginx location block.